

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS Marks: 50			Total
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with Prepared Statement.	BL3 – Apply	5

4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I D. Gnana Deepthi(192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D. Gnanadeepthi.

ANSWERS

1. Query Execution Plan and Heuristic Optimization

Given Complex SQL Query

```
SELECT E.EmpName, D.DeptName, P.ProjectName
FROM Employee E
JOIN Department D ON E.DeptID = D.DeptID
JOIN Works_On W ON E.EmpID = W.EmpID
JOIN Project P ON W.ProjectID = P.ProjectID
WHERE E.Salary > 50000
      AND D.Location = 'Chennai'
      AND P.Budget > 100000
      AND E.EmpID IN (
        SELECT EmpID
        FROM Attendance
        WHERE DaysPresent > 20
      );
```

1. Initial Query Execution Plan

The DBMS first creates a relational algebra expression:

```
[
  \pi_{EmpName,DeptName,ProjectName}
  \left(
    \sigma_{Salary>50000 \wedge Location='Chennai' \wedge Budget>100000 \wedge
    DaysPresent>20}
    \left(
      Employee \bowtie Department \bowtie Works_On \bowtie Project \bowtie Attendance
    \right)
  \right)
]
```

Initial plan:

PROJECT

```

      |
SELECT
      |
      JOIN
    /  \
  JOIN   Attendance
 /  \
JOIN   Project
 /  \
JOIN   Works_On
 /  \

```

Employee Department

This plan is inefficient because it performs large joins before reducing the number of records.

2. Heuristic Optimization Rules Applied

Rule 1: Perform Selection Operations Early

Apply WHERE conditions directly to individual relations:

σ Salary > 50000 (Employee)

σ Location = 'Chennai' (Department)

σ Budget > 100000 (Project)

σ DaysPresent > 20 (Attendance)

This reduces the number of tuples before joining.

Employee Department Project Attendance

```

      |           |           |           |

```

σ Salary>50000 σ Location='Chennai' σ Budget>100000 σ DaysPresent>20

Rule 2: Convert Subquery into a Join

The nested subquery:

E.EmpID IN (

```

SELECT EmpID
FROM Attendance
WHERE DaysPresent > 20
)

```

can be transformed into a **join or semijoin**:

Employee_filtered $\bowtie$ Attendance_filtered

Only employees having attendance greater than 20 days are retained.

Rule 3: Perform Projections Early

Only required attributes are kept.

Employee: EmpID, EmpName, DeptID

Department: DeptID, DeptName

Works_On: EmpID, ProjectID

Project: ProjectID, ProjectName

Attendance: EmpID

This reduces memory and disk I/O.

Rule 4: Join Small/Filtered Relations First

First join the filtered Employee relation with filtered Attendance:

```

[
Employee_{filtered} \bowtie Attendance_{filtered}
]

```

Then join with filtered Department:

```

[
(Employee \bowtie Attendance) \bowtie Department
]

```

Next join with Works_On:

```

[
((Employee \bowtie Attendance) \bowtie Department) \bowtie Works_On
]

```

Finally join with filtered Project:

```
[
(((Employee \bowtie Attendance) \bowtie Department)
\bowtie Works_On) \bowtie Project
]
```

3. Optimized Query Execution Plan

```

      π EmpName, DeptName, ProjectName
      |
      JOIN
      /  \
      JOIN   π ProjectID, ProjectName
      /  \   |
      JOIN   Works_On  σ Budget > 100000
      /  \   |
      SEMIJOIN Department   Project
      /  \   |
      Employee Attendance  σ Location='Chennai'
      |       |           |
      σ Salary>50000  σ DaysPresent>20 Department

```

Step-by-Step Optimized Plan

1. **Filter Employee** where Salary > 50000.
2. **Filter Department** where Location = 'Chennai'.
3. **Filter Project** where Budget > 100000.
4. **Filter Attendance** where DaysPresent > 20.
5. Replace the nested IN subquery with a **semijoin** between Employee and Attendance.
6. Apply **projection** to keep only required attributes.
7. Join the smallest filtered intermediate relations first.
8. Perform the final projection to display EmpName, DeptName, and ProjectName.

2. Cost-Based Optimization for Join Algorithms

Consider two relations **R** and **S** with the following details:

Relation	Number of Tuples	Number of Pages
----------	------------------	-----------------

R	10,000	500 pages
---	--------	-----------

S	5,000	200 pages
---	-------	-----------

Assume the available buffer memory is sufficient for **hash join**, and the cost is measured in **disk I/O operations**.

1. Nested-Loop Join (NLJ)

The cost of nested-loop join is:

$$\begin{aligned} &[\\ \text{Cost} &= B(R) + B(R) \times B(S) \\ &] \end{aligned}$$

Where:

- $(B(R) = 500)$ pages
- $(B(S) = 200)$ pages

$$\begin{aligned} &[\\ \text{Cost} &= 500 + (500 \times 200) \\ &] \end{aligned}$$

$$\begin{aligned} &[\\ &= 500 + 100000 = 100500 \\ &] \end{aligned}$$

Nested-Loop Join Cost = 100,500 I/Os

2. Sort-Merge Join (SMJ)

If both relations are not already sorted, they must first be sorted.

Approximate sorting cost:

$$\begin{aligned} &[\\ &2B \log_2 B \\ &] \end{aligned}$$

For relation R:

$$[2(500) \log_2(500) \approx 8966]$$

For relation S:

$$[2(200) \log_2(200) \approx 3058]$$

After sorting, the relations are merged:

$$[500 + 200 = 700]$$

Therefore:

$$[\text{Cost}(\text{SMJ}) = 8966 + 3058 + 700]$$

$$[= 12724]$$

Sort-Merge Join Cost $\approx 12,724$ I/Os

3. Hash Join

For a hash join, the approximate cost is:

$$[\text{Cost} = 3(B(R) + B(S))]$$

$$[= 3(500 + 200)]$$

$$[= 3(700)]$$

$$[= 2100]$$

Hash Join Cost = 2,100 I/Os

4. Cost Comparison

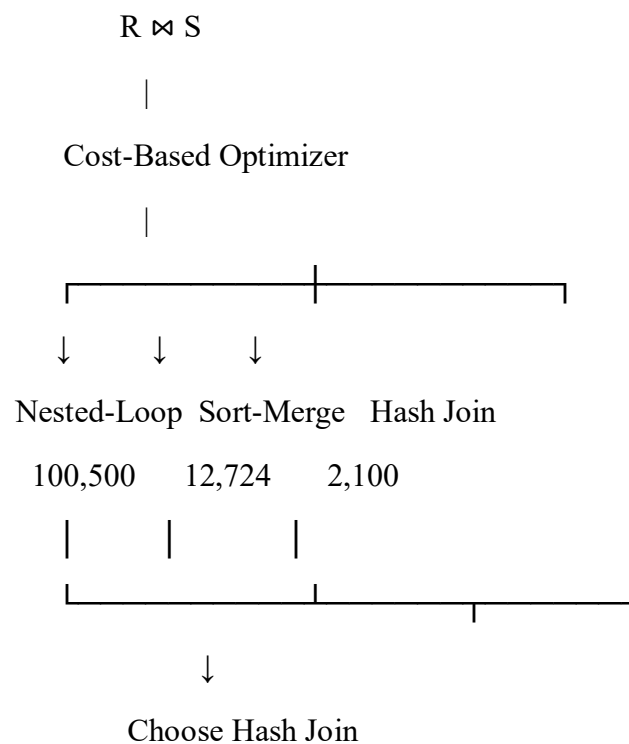
Join Method Cost (Disk I/O)

Nested-Loop Join 100,500

Sort-Merge Join $\approx 12,724$

Hash Join **2,100**

5. Query Execution Plan



3. Implement JDBC Connectivity in Java with MySQL and Prepared Statement

Aim

To establish JDBC connectivity between a Java program and a MySQL database and execute a parameterized query using PreparedStatement.

Requirements

- Java JDK

- MySQL Server
- MySQL Connector/J driver

Database Setup

```
CREATE DATABASE college_db;

USE college_db;

CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    department VARCHAR(50),
    cgpa DECIMAL(3,2)
);

INSERT INTO student VALUES
(1, 'Deepthi', 'CSE', 8.71),
(2, 'Varun', 'IT', 9.23),
(3, 'Aadhya', 'ECE', 8.97);
```

Java Program

```
import java.sql.*;

public class JDBCPreparedStatementDemo {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college_db";

        String username = "root";

        String password = "your_password";

        // Parameterized SQL query

        String query = "SELECT * FROM student WHERE department = ? AND cgpa >= ?";

        try {
```

1. Load MySQL JDBC Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Establish connection

```
Connection con = DriverManager.getConnection(url, username, password);
```

```
System.out.println("Database connected successfully!");
```

3. Create PreparedStatement

```
PreparedStatement ps = con.prepareStatement(query);
```

4. Set parameter values

```
ps.setString(1, "CSE");
```

```
ps.setDouble(2, 8.0);
```

5. Execute query

```
ResultSet rs = ps.executeQuery();
```

6. Display results

```
System.out.println("\nStudent Details:");
```

```
while (rs.next()) {  
    System.out.println(  
        "ID: " + rs.getInt("id") +  
        ", Name: " + rs.getString("name") +  
        ", Department: " + rs.getString("department") +  
        ", CGPA: " + rs.getDouble("cgpa")  
    );  
}
```

7. Close resources

```
rs.close();
```

```
ps.close();
```

```
con.close();
```

```
} catch (ClassNotFoundException e) {
```

```
        System.out.println("JDBC Driver not found!");

    } catch (SQLException e) {

        System.out.println("Database error: " + e.getMessage());

    }

}

}
```

Expected Output

Database connected successfully!

Student Details:

ID: 1, Name: Deepthi, Department: CSE, CGPA: 8.71

Working

1. Class.forName() loads the MySQL JDBC driver.
2. DriverManager.getConnection() connects Java to the MySQL database.
3. PreparedStatement is used for parameterized SQL queries.
4. ? represents placeholders for values.
5. setString() and setDouble() provide values safely.
6. executeQuery() executes the SELECT statement.
7. ResultSet retrieves and displays the records.

4. Conflict-Serializability Using a Precedence Graph

Given Schedule

Consider the following transactions:

- **T1:** R1(X), W1(X)
- **T2:** R2(X), W2(Y)
- **T3:** R3(Y), W3(X)
- **T4:** R4(Y), W4(Y)

Assume the schedule is:

$R1(X) \rightarrow R2(X) \rightarrow W1(X) \rightarrow R3(Y) \rightarrow W2(Y) \rightarrow R4(Y) \rightarrow W3(X) \rightarrow W4(Y)$

Step 1: Identify Conflicting Operations

Two operations conflict when:

1. They belong to **different transactions**.
2. They access the **same data item**.
3. At least one operation is a **write**.

Conflicts on X

- $R2(X)$ occurs before $W1(X) \rightarrow T2 \rightarrow T1$
- $W1(X)$ occurs before $W3(X) \rightarrow T1 \rightarrow T3$
- $R1(X)$ occurs before $W3(X) \rightarrow T1 \rightarrow T3$

Conflicts on Y

- $R3(Y)$ occurs before $W2(Y) \rightarrow T3 \rightarrow T2$
- $W2(Y)$ occurs before $R4(Y) \rightarrow T2 \rightarrow T4$
- $W2(Y)$ occurs before $W4(Y) \rightarrow T2 \rightarrow T4$
- $R4(Y)$ occurs before $W4(Y) \rightarrow T4 \rightarrow T2$

Step 2: Construct the Precedence Graph

Each transaction is represented as a node.

An edge $T_i \rightarrow T_j$ means that transaction T_i must be executed before T_j .

$T2 \longrightarrow T1$

$\uparrow \quad |$

$| \quad \downarrow$

$T3 \longleftarrow$

$T2 \longrightarrow T4$

$\uparrow \quad |$

$\longleftarrow$

Edges

$T2 \rightarrow T1$

T1 → T3

T3 → T2

T2 → T4

T4 → T2

Step 3: Check for Cycles

The graph contains cycles:

T1 → T3 → T2 → T1

and

T2 → T4 → T2

Since the precedence graph contains a **cycle**, the schedule is **not conflict-serializable**.

5. Demonstrate Basic Two-Phase Locking (2PL) and Resolve Deadlock

Given Transactions

Consider two transactions:

T1: Read(A) → Write(A) → Read(B) → Write(B)

T2: Read(B) → Write(B) → Read(A) → Write(A)

Since both transactions modify data items, **exclusive locks (X-locks)** are required.

Step 1: Apply Basic 2PL

Under Basic 2PL, each transaction has two phases:

1. **Growing Phase:** The transaction can acquire locks but cannot release any lock.
2. **Shrinking Phase:** The transaction can release locks but cannot acquire any new lock.

Locking Schedule

Step	Transaction	Operation	Result
1	T1	XL(A)	Granted
2	T1	Read(A), Write(A)	Executed

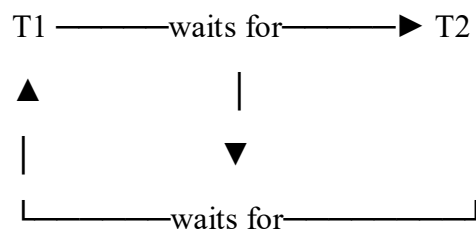
	Step	Transaction	Operation	Result
	3	T2	XL(B)	Granted
	4	T2	Read(B), Write(B)	Executed
	5	T1	XL(B)	Waiting – B is locked by T2
	6	T2	XL(A)	Waiting – A is locked by T1

Step 2: Identify the Deadlock

At this stage:

- **T1 holds A** and waits for **B**.
- **T2 holds B** and waits for **A**.

Wait-for Graph



Or:

$T1 \rightarrow T2 \rightarrow T1$

Since the wait-for graph contains a **cycle**, a **deadlock occurs**.

Step 3: Resolve the Deadlock

The DBMS can select one transaction as the **victim** and roll it back.

Assume **T2 is selected as the victim**.

Resolution

1. Abort **T2**.
2. Release lock XL(B).
3. Grant XL(B) to **T1**.
4. T1 completes its operations and commits.
5. T1 releases locks on A and B.
6. Restart T2.

7. T2 obtains the required locks and completes successfully.

Resolved Execution

T1: XL(A) → R(A) → W(A) → XL(B) → R(B) → W(B) → COMMIT → UNLOCK(A),
UNLOCK(B)

T2: XL(B) → R(B) → W(B) → waits for XL(A)

↓

Deadlock detected

↓

ROLLBACK T2 → UNLOCK(B)

↓

Restart T2 after T1 commits

2PL Phases

T1

Growing Phase:

XL(A) → XL(B)

Shrinking Phase:

UNLOCK(A) → UNLOCK(B)

T2

Growing Phase:

XL(B) → XL(A)

Shrinking Phase:

UNLOCK(A) → UNLOCK(B)

6. Apply Wait-Die and Wound-Wait Deadlock Prevention Schemes

Given Transactions

Assume the following timestamps:

Transaction Timestamp Age

T1	10	Oldest
T2	20	
T3	30	Youngest

Smaller timestamp = older transaction.

Assume the following lock requests:

1. **T2 locks A**
2. **T3 locks B**
3. **T1 requests A**
4. **T2 requests B**
5. **T3 requests A**

1. Wait-Die Scheme

Rule

- Older transaction requests a lock held by a younger transaction → Wait
- Younger transaction requests a lock held by an older transaction → Abort (Die) and restart later

Step-by-Step Execution

Step	Lock Request	Decision	Reason
1	T2 locks A	Granted	A is free
2	T3 locks B	Granted	B is free
3	T1 requests A	Wait	T1 is older than T2
4	T2 requests B	Wait	T2 is older than T3
5	T3 requests A	Die / Abort	T3 is younger than T2

Result

T1 (older) → waits for T2

T2 (older) → waits for T3

T3 (younger) → dies because it requests A held by older T2

After T3 aborts, it releases lock B. Then T2 obtains B, completes, and releases A. Finally, T1 obtains A and completes.

Wait-Die Conclusion

Older transaction → WAIT

Younger transaction → DIE (ABORT)

2. Wound-Wait Scheme

Rule

- Older transaction requests a lock held by a younger transaction → Abort the younger transaction (Wound)
- Younger transaction requests a lock held by an older transaction → Wait

Step-by-Step Execution

	Step	Lock Request	Decision	Reason
1	T2 locks A	Granted	A is free	
2	T3 locks B	Granted	B is free	
3	T1 requests A	Wound T2	T1 is older than T2	
4	T2 requests B	T2 was aborted	Lock request cancelled	
5	T3 requests A	Wait	T3 is younger than T2/T1	

After T2 is aborted:

1. T2 releases lock A.
2. T1 obtains lock A.
3. T1 completes and releases A.
4. T3 then obtains A and completes.
5. T2 can restart later with its original timestamp.

Comparison of Wait-Die and Wound-Wait

Scheme	Older Requests Younger's Lock	Younger Requests Older's Lock
---------------	--------------------------------------	--------------------------------------

Wait-Die	Wait	Abort (Die)
-----------------	------	-------------

Wound-Wait	Abort younger (Wound)	Wait
-------------------	-----------------------	------

7. Immediate Update Recovery Protocol Using Transaction Logs and Checkpoints

Given Transaction Log

Assume the following transaction log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<CHECKPOINT {T2}>

<START T3>

<T3, C, 300, 350>

<T2, D, 400, 450>

<COMMIT T3>

<START T4>

<T4, E, 500, 550>

<SYSTEM FAILURE>

The log record format is:

```
[
<T_i,\ DataItem,\ OldValue,\ NewValue>
]
```

Step 1: Identify the Last Checkpoint

The last checkpoint is:

<CHECKPOINT {T2}>

At this checkpoint:

- **T1 has already committed.**
- **T2 is active.**
- Transactions starting after the checkpoint are **T3** and **T4**.

Step 2: Identify Committed and Uncommitted Transactions

Committed Transactions — REDO

Transaction Status

T1	Committed before checkpoint
T3	Committed
T2	Not committed
T4	Not committed

So:

[
REDO = {T3}
]

Uncommitted Transactions — UNDO

[
UNDO = {T2, T4}
]

T1 does not need recovery action because it committed before the checkpoint.

Step 3: Apply REDO Operation

Transaction **T3** committed before the system failure.

Log record:

<T3, C, 300, 350>

Therefore, REDO:

C = 350

This ensures that the committed update of T3 is reflected in the database.

Step 4: Apply UNDO Operations

Undo T4

Log record:

<T4, E, 500, 550>

Restore the old value:

E = 500

Undo T2

T2 did not commit. Its updates are undone:

<T2, D, 400, 450> → D = 400

<T2, B, 200, 250> → B = 200

Undo is generally performed in **reverse order**.

Step 5: Final Database State

Data Item Final Value

A	150
B	200
C	350
D	400
E	500

Recovery Process

System Failure

|

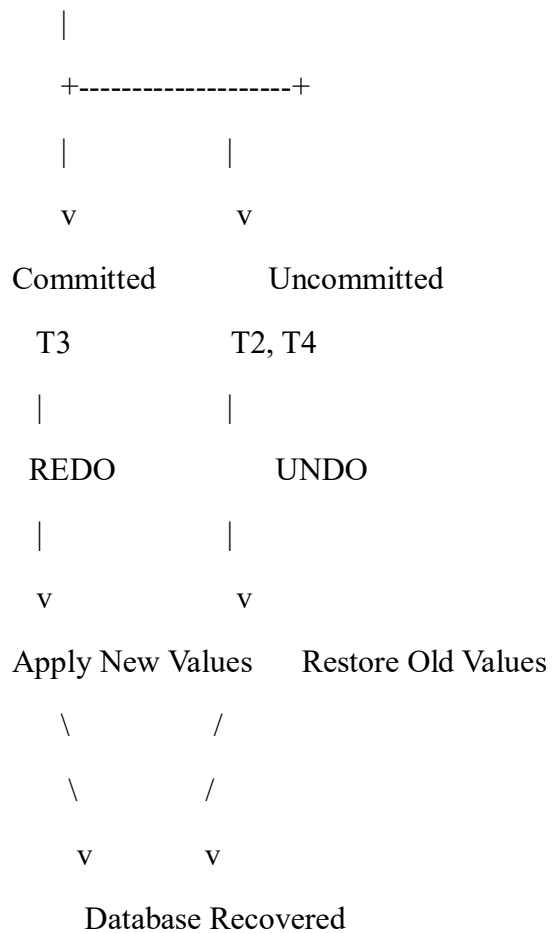
v

Find Last Checkpoint

|

v

Identify Transactions



8. Deferred Update (NO-UNDO/REDO) Recovery Technique

Given Log File

Assume the following transaction log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<START T3>

<T3, C, 300, 350>

<COMMIT T3>

<T2, D, 400, 450>

<START T4>

<T4, E, 500, 550>

<SYSTEM FAILURE>

Log record format:

```
[  
<T_i, \ DataItem, \ OldValue, \ NewValue>  
]
```

Step 1: Identify Committed and Uncommitted Transactions

Committed Transactions

Transaction Status

T1 Committed

T3 Committed

Therefore:

```
[  
REDO = {T1,T3}  
]
```

Uncommitted Transactions

Transaction Status

T2 Not committed

T4 Not committed

Therefore:

```
[  
NO\ UNDO = {T2,T4}  
]
```

Step 2: Apply Deferred Update Recovery Rule

In the **Deferred Update** technique, database updates are **not written to the actual database until the transaction commits**.

Therefore:

- **Committed transactions → REDO**
- **Uncommitted transactions → No action**
- **UNDO is not required**

This is why it is called:

```
[  
\boxed{NO\text{-}UNDO/REDO}  
]
```

Step 3: REDO the Committed Transactions

Transaction T1

<T1, A, 100, 150>

Apply the new value:

```
[  
A = 150  
]
```

Transaction T3

<T3, C, 300, 350>

Apply the new value:

```
[  
C = 350  
]
```

Step 4: Ignore Uncommitted Transactions

Transaction T2

<T2, B, 200, 250>

<T2, D, 400, 450>

T2 did not commit, so its changes were **not written to the database**.

Transaction T4

<T4, E, 500, 550>

T4 also did not commit, so its changes are ignored.

Thus:

[
T2,T4 \rightarrow \text{NO UNDO required}
]

Step 5: Final Database State

Data Item Final Value

A	150
B	200
C	350
D	400
E	500

Recovery Process

SYSTEM FAILURE

|

v

Examine Transaction Log

|

+-----+-----+

|

|

v

v

Committed

Uncommitted

T1, T3

T2, T4

|

|

v

v

REDO

NO ACTION

|

v

Apply New Values

|

v

Database Recovered

9. Design a Concurrency Control Mechanism Using Strict 2PL for a Banking System

Given Transactions

Assume two accounts:

- **Account A** = Source account
- **Account B** = Destination account

Transaction T1 – Transfer Funds

T1 transfers ₹1,000 from Account A to Account B.

T1:

Read(A)

$A = A - 1000$

Write(A)

Read(B)

$B = B + 1000$

Write(B)

Commit

Transaction T2 – Read Balance

T2 reads the balance of Account A.

T2:

Read(A)

Display(A)

Commit

Strict 2PL Rules

Strict 2PL uses the following locking mechanism:

- **Shared Lock (S-lock):** Required for reading.
- **Exclusive Lock (X-lock):** Required for writing.
- A transaction cannot release its **exclusive locks until it commits or aborts**.
- This prevents other transactions from reading uncommitted data.

Step 1: T1 Starts the Fund Transfer

T1 needs exclusive locks because it updates both accounts.

T1: XLOCK(A) → Granted

T1: Read(A)

T1: $A = A - 1000$

T1: Write(A)

T1: XLOCK(B) → Granted

T1: Read(B)

T1: $B = B + 1000$

T1: Write(B)

At this point, T1 holds:

XLOCK(A), XLOCK(B)

Step 2: T2 Attempts to Read Account A

T2 requests a shared lock:

T2: SLOCK(A)

However, Account A is already locked exclusively by T1.

Therefore:

T2 → WAIT

T2 cannot read A until T1 completes.

Step 3: T1 Commits

After completing the transfer:

T1: COMMIT

T1: UNLOCK(A)

T1: UNLOCK(B)

Under **Strict 2PL**, T1 keeps its exclusive locks until the commit operation.

Step 4: T2 Reads the Updated Balance

After T1 releases the lock:

T2: SLOCK(A) → Granted

T2: Read(A)

T2: Display(A)

T2: COMMIT

T2: UNLOCK(A)

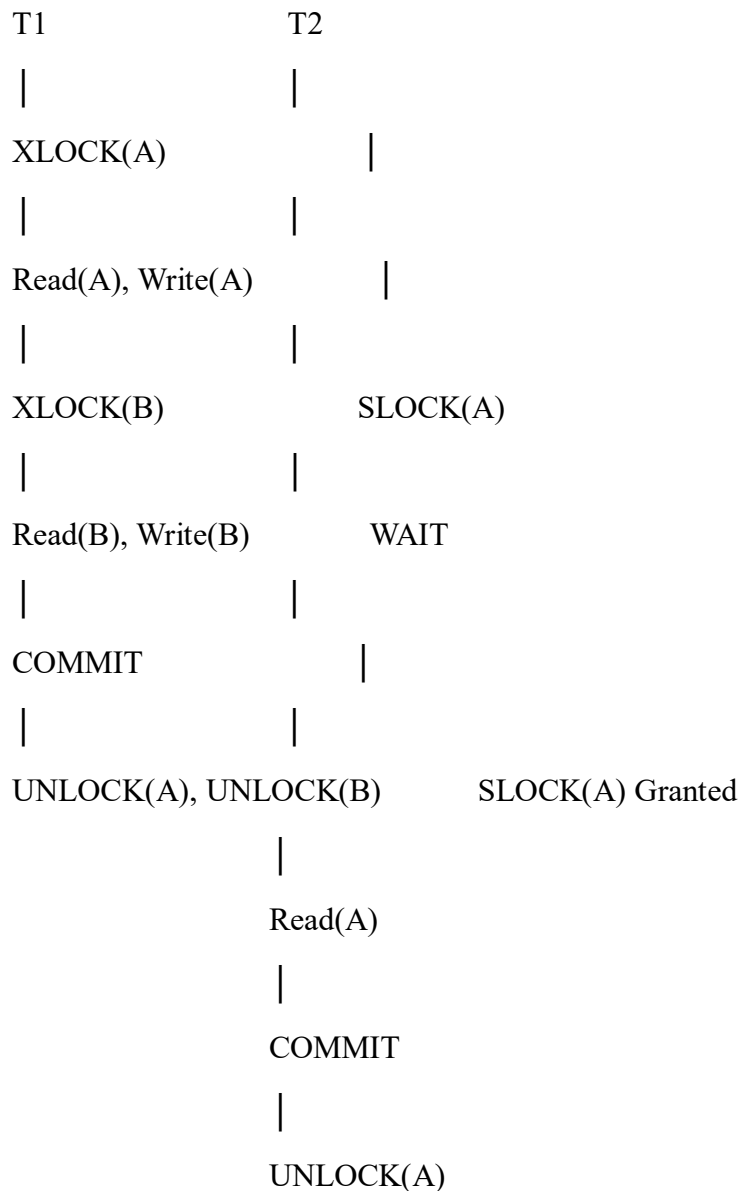
Thus, T2 reads only the **committed and updated balance**.

Locking Schedule

Step T1 – Transfer Transaction T2 – Read Transaction

1	XLOCK(A) – Granted	
2	Read(A), Write(A)	
3	XLOCK(B) – Granted	
4	Read(B), Write(B)	SLOCK(A) – Waiting
5	COMMIT	Waiting
6	UNLOCK(A), UNLOCK(B)	SLOCK(A) – Granted
7		Read(A), Display(A)
8		COMMIT, UNLOCK(A)

Execution Flow



Benefits of Strict 2PL in Banking

1. Prevents **dirty reads** because T2 cannot read uncommitted data.
2. Ensures **conflict serializability**.
3. Prevents **cascading rollback**.
4. Maintains **database consistency** during fund transfers.
5. Ensures T2 reads the correct committed balance.

10. Pseudocode for Query Optimization Using Heuristics

Pseudocode

```
ALGORITHM Heuristic_Query_Optimization(QueryPlan)
BEGIN
    // Step 1: Push down selection operations
    FOR each Selection node S in QueryPlan DO
        WHILE S can be moved closer to its related base relation DO
            Move S downward in the query tree
        END WHILE
    END FOR

    // Step 2: Apply projection early
    FOR each Relation R in QueryPlan DO
        RequiredAttributes ← attributes needed for:
            SELECT output,
            JOIN conditions,
            WHERE conditions

        Replace R with PROJECT(RequiredAttributes, R)
    END FOR

    // Step 3: Estimate size of intermediate results
    FOR each possible Join J in QueryPlan DO
        Estimate J.result_size
    END FOR

    // Step 4: Order joins by smaller intermediate results
    Sort all joins in ascending order of estimated result_size

    OptimizedPlan ← first relation/join with smallest result

    FOR each remaining Join J in sorted order DO
```

```

    OptimizedPlan ← JOIN(OptimizedPlan, J)
END FOR

// Step 5: Return the optimized query plan
RETURN OptimizedPlan
END

```

Explanation of Heuristics

1. Push Down Selection

Selection conditions are applied as close as possible to the original relation.

Before:

```

    σ condition
    |
    JOIN
    /  \
   R    S

```

After:

```

    JOIN
    /  \
σ condition  S
|
R

```

This reduces the number of tuples before the join operation.

2. Project Early

Only the attributes required for the final output, joins, and conditions are retained.

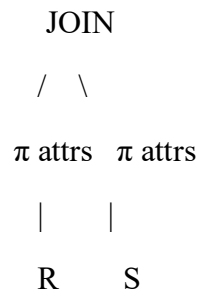
Before:

```

    JOIN
    /  \
   R    S

```

After:



This reduces the size of intermediate results.

3. Order Joins by Smaller Intermediate Results

The optimizer estimates the size of each possible join and performs the join producing the **smallest intermediate result first**.

Example:

Join(R1, R2) → 500 tuples

Join(R2, R3) → 100 tuples

Join(R1, R3) → 800 tuples

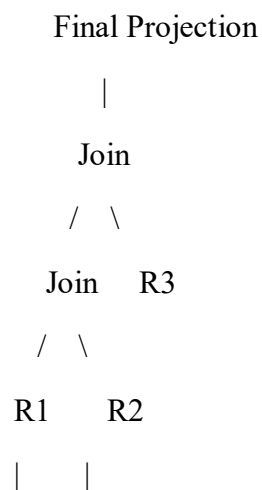
Preferred order:

JOIN(R2, R3)

↓

JOIN(result, R1)

Optimized Query Plan



Selection Selection

| |

Projection Projection

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided